

RM dev environment setup

1. [Enabling RM debug tools](#)
2. [Setting hotkeys](#)
3. [Getting started](#)
4. [XS debugger](#)
5. [RM visualizer](#)
6. [Specific match settings](#)
7. [Other useful configurations](#)

Random maps can be debugged and generated in the editor, but it is often more convenient to develop in skirmish when testing more “complex” player combinations (e.g., FFA, 1v11, 2v2v2v2v2v2, ...) or specific game modes. The instructions below work for either setup, unless stated otherwise.

1. Enabling RM debug tools

For this step to be possible we need to first understand hotkeys + configs and have the related files set up. Read and follow “*BANG documentation\Config documentation\Config basics*” and “*BANG documentation\Hotkey documentation\Hotkey basics*”.

You should now have a *user.con* and *user.cfg* and understand how to use them.

The relevant config for RM development is *debugRandomMaps*. This config by itself will enable all actual debug tools for us. There are more RM-related configs, which can be found lower on this page.

2. Setting hotkeys

Although optional, setting some hotkeys is strongly recommended to make full use of the dev tools.

Add the following lines to your *user.con* file, and make sure you adjust the hotkeys you want to use accordingly.

Command (goes into <i>user.con</i>)	Description
<code>map("*", "game", "leaveGame()")</code>	Leaves the game immediately, going back to the main menu. Useful when testing from skirmish – make sure to not set this to something you can accidentally hit while playing a normal game!
<code>map("numpadminus", "game", "restartCurrentGame()")</code>	Restarts the current game with the same seed (i.e., the generated map will look the same). Your bread and butter when testing and (especially) when debugging RMs in skirmish.
<code>map("numpadplus", "game", "restartCurrentGameWithSeed(-1)")</code>	Same as above, except that the current game is restarted with a different seed, meaning you have the same player configuration available, but roll a new map generation.
<code>map("numpaddivide", "root", "gadgetToggle(\"XSDebugger\")")</code>	Toggles the XS debugger. This will be one of your two best friends when developing random map scripts, so make sure you set this.
<code>map("numpad9", "world", "toggleRMVisualizer()")</code>	Toggles the RM visualizer. This will be your other best friend when developing random map scripts.

3. Getting started

To verify our *debugRandomMaps* config was correctly enabled and our hotkeys work we will run through all the debug tools while in-game:

1. Restart your game to have the *debugRandomMaps* config take effect.
2. Either: (You can also try out both workflows, of course!)
 - a. Go to the scenario editor, click “New” in the top left corner, and generate any map.
 - b. Go to skirmish, fill in a player combination of your liking, set the map visibility to “Visible”, and start the game
3. Your map generated, so it’s time to try out the hotkeys you set above – feel free to try them all, but we will have a particular look at the XS debugger and the RM visualizer.

4. XS debugger

First, make sure you read the page of the XS debugger, which can be found here: “*BANG documentation\XS documentation\XS debugger*”

For random maps, the proper runtime should already be selected after generating a map, and so you should see the script that was used for generation on the left side.

Breakpoints: At times, you may want to pause generation to step through code as the script is running. Since the script compiles and runs in one go, at least the initial breakpoint is usually script-driven. To trigger a breakpoint and show the debugger while the script is running you can use any of the following syscalls in your script:

- `rmEchoWarning("your warning message")`
- `rmEchoError("your error message")`

Once a breakpoint has been triggered, you can use the XS debugger for RMs as described in the above document.

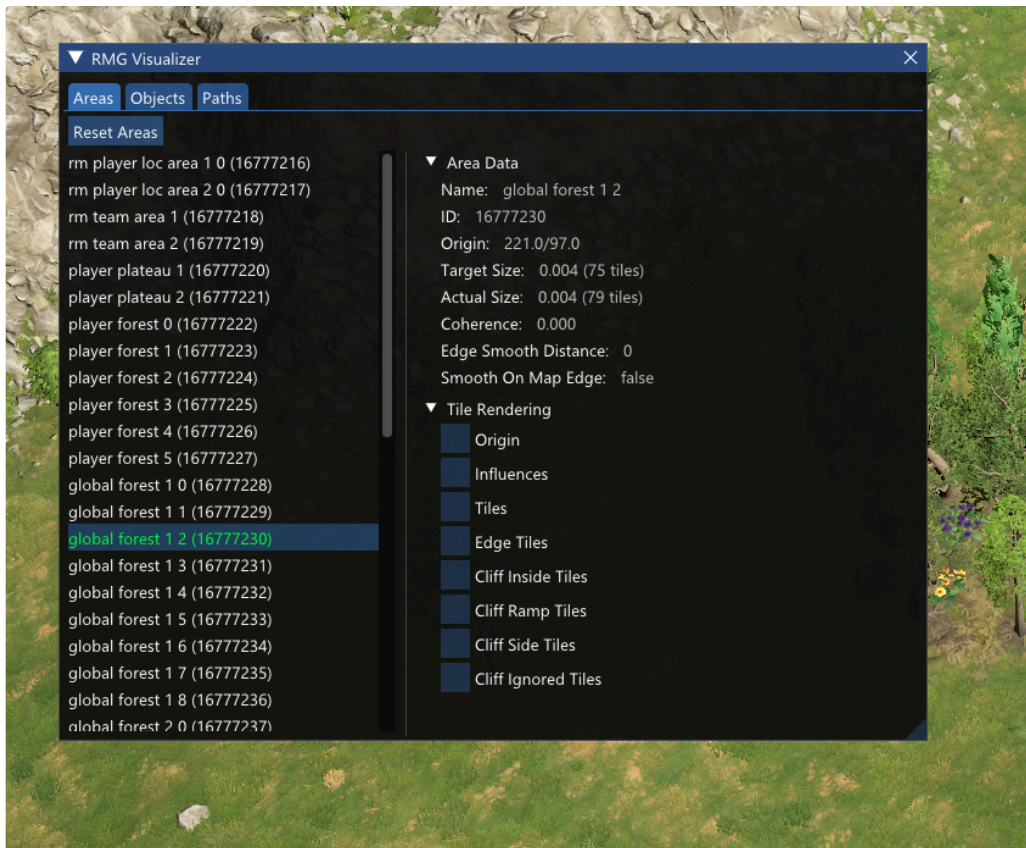
However, RM development typically makes heavy usage of the output window of the right side – the generation log printed when generating a map. The log shows all messages a script prints (and also some engine-related messages), so a common practice is to quickly show/hide the debugger after map generation if the output is of interest.

5. RM visualizer

The RM visualizer is new, small, and very powerful tool for RM development in Retold. It can be toggled by using the hotkey for the RM visualizer you set earlier, and will bring up a small ImGui window with multiple tabs for areas, objects, and paths.

The visualizer renders the tiles these entities occupy, along with some other information. Aside of the checkboxes that can be toggled for rendering, the following clicking behavior is used:

- Single click: Selection
- Double click: Enable rendering for the entity and move the camera to the location of the entity (or its center).
- Right click: Unrender an entity.
- Middle mouse button: Jump to the location of an entity (or its center) without enabling rendering.



RM Visualizer (here featuring the Areas tab).

6. Specific match settings

Random maps should always account for any player or team setup as good as possible. Naturally, this has to be tested in one way or another. While setting up the specific settings in a skirmish lobby and then starting from there, this can be tedious, especially as there are many different combinations.

Thus, it makes sense to have a shortcut for the most popular configurations.*

The following binding can be used to generate normal sized 1v1, 2v2, etc. maps. Note that this ONLY works for the map file specified – which means it can make sense to develop or move a script into said file while working on it for the sake of production speed.

```
map("numpad1", "world", "startRandomGame2(\"0.xs\", 2, 2, -1, 0)")
```

0.xs in this case is the script name (starting with 0 can be useful as this means it will appear at the top of any list), followed by the number of players (2), number of teams (2), seed (-1 means random), and map size (0 means standard).

*Ideally, this would be solved through an editor setting where diplomacy and player count can be set before generating the map (and the setting persisting after generation so that more maps with the same settings can be quickly generated).

7. Other useful configurations

There is a number of additional useful configurations for RM development:

Configuration (goes into <i>user.cfg</i>)	Description
<code>noIntroCinematics</code>	Disables the intro cinematics to get faster to the main menu.

<code>disableAssetPreloading</code>	Disables asset preloading to reduce RM load times.
<code>generateRMConstants</code>	Like the other runtimes, a list of exported RM constants can be generated and found in the user folder under <i>temp/Logs/MythRMConstants.txt</i>. This includes variables like the number of players or teams, lobby settings, or terrain IDs.